Certificate-Gated Neural Proposers for Verifier-Grounded Action Scoring

PLACEHOLDER: Authors and affiliations

Working draft — ideal-paper roadmap

Abstract. When an exact verifier is available, the right architecture for action scoring is not an end-to-end neural policy. We specify one reusable verifier-gated action task schema: complete candidate enumeration, observable state/action features, a neural residual scorer, an exact verifier, a symbolic rule grammar, a certificate checker, a rule proposer, and a trace/replay/RL protocol. We instantiate this schema on four certified tasks: prime gaps, grid paths, delayed claims, and ARC-AGI. Prime gaps currently provide the strongest apples-to-apples gate ablation: with residue-sieve features removed from the scorer input, the raw scorer reaches 9.13% top-1; a fixed hand-wheel gate raises this to 45.12%, and a learned rule gate reaches 68.87% at 100% top-10. Grid paths show the same certified-gate pattern outside arithmetic: with Manhattan priors disabled, a budgeted legality gate raises accuracy from 57.40% to 68.90%; with priors enabled the same gate lifts 75.80% raw to 89.60% gated. Delayed claims currently exercise trace replay and calibration (parent 93.50% / replay successor 94.80%), not yet gate learning. On the canonical ARC-AGI-1 task `0520fde7`, a 28.56% top-1 neural cell scorer is replaced by a certified transform gate that emits the unique grammar-consistent prediction at 100% top-1 on all nine held-out cells; on the public 400-task ARC training corpus the same gate certifies a 24/400 floor under grammar v10, and a thin `NNHypothesis` slot-induction layer adds transform families v10 cannot synthesise. The prime rule-search negative result is load-bearing: grammar order and a simple structural heuristic beat the current neural proposer at a fixed 16-rule budget, and the conditional gap grammar exposes a larger failure. A separate `program_synthesis` cross-port emits nine certified `crlp.hypothesis.evidence.v1` rows through the same library schema, demonstrating that the substrate is not ARC-specific. The contribution is therefore the generalized protocol and audit discipline, not an improved prime algorithm or a competitive ARC solver.

1. Introduction

Exact verifiers are common in domains where action correctness can be checked algorithmically even when discovery cannot: theorem proving, program synthesis, combinatorial search, formal shortest-path problems. The dominant style in modern learned policy work treats the verifier as a reward signal for an end-to-end neural policy. We argue this is the wrong default when the candidate space contains many structurally impossible actions that the verifier can cheaply eliminate.

Our thesis is the **certificate-gating loop**: a learned scorer should consume the **survivors** of a verifier-checked symbolic filter, and the filter itself can be produced by a rule proposer gated by a soundness-certificate verifier. The scorer ranks the residual; the gate eliminates the rest; nothing the gate emits can violate correctness because every rule it accepts carries a deterministic proof.

We make three protocol claims, each of which must be instantiated across tasks before being treated as general:

(C1) **Certified gates reduce the residual action space.** The complete prime-gap result and the smaller grid-path legality result support this. Delayed-claim gate learning remains to be implemented.

(C2) **Rule search must beat fixed-budget symbolic baselines.** The current prime result is negative for neural proposer learning: grammar order beats the neural proposer on the unconditional candidate grammar, and the conditional gap grammar makes the failure larger. This turns proposer learning into a concrete cross-task benchmark rather than a tautology.

(C3) **Verifier RL must preserve calibration across tasks.** Strict policy gradient on sparse exact rewards inflates logit scale on prime gaps; a standard KL-to-prior + entropy term restores calibration at unchanged top-k. The same run is still needed on grid paths and delayed claims.

The artifact now emits `crlp.experiment_schema.v1` rows for every certified task with `make experiment-schemas`. The paper should be read against that schema: task-specific verifiers and grammars change, but candidate enumeration, leakage discipline, proposer baselines, trace replay, and verifier-RL comparisons are shared.

2. Related Work

PLACEHOLDER — Required citations and 1-paragraph positioning against each:

- Process supervision (Lightman et al., 2023): we use a similar verifier-as-step-checker pattern; their reward model is learned, ours is exact-by-construction.
- Verifier-guided decoding / best-of-N (Cobbe et al., 2021; Hosseini et al., 2024): we gate **before** scoring, not after.
- AlphaProof / AlphaGeometry symbolic-neural search (Trinh et al., 2024): our rule synthesis is structurally similar but verifier-grounded by certificate rather than by proof completion.
- RLHF KL-regularized policy gradient (Stiennon et al., 2020; Ouyang et al., 2022): Axis 3 applies this objective to exact-verifier rewards.
- Neural-symbolic program synthesis (NeSy literature).

3. Method

3.1. The Certificate-Gating Loop

The reusable architecture is:

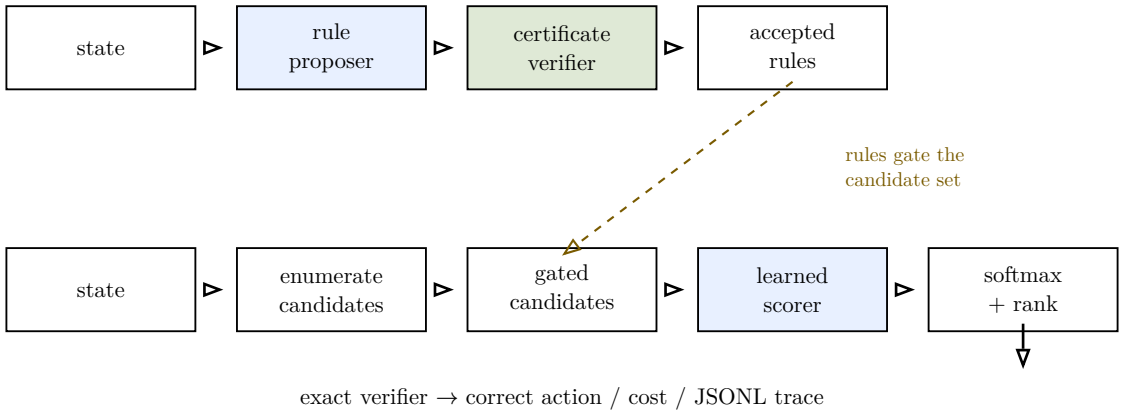


Figure 1: The certificate-gating loop. A rule proposer (top) produces candidate rules that a soundness-certificate verifier accepts or rejects. Accepted rules deterministically prune the candidate set; a shared neural scorer ranks the survivors; the exact verifier supplies the correctness signal.

The design invariant is **the model proposes, the verifier decides, the trace remembers.** A rule r is accepted only if a deterministic algebraic check certifies that r cannot reject the true action on any state in the support set, so soundness does not depend on the proposer being well-trained.

3.2. Generalized Task Schema

Every certified experiment in the artifact emits the same schema row:

```
Task T = {
  state distribution S_T,
  candidate enumerator A_T(s),
  observable features x_T(s, a),
  neural residual scorer f_theta(x_T),
  exact verifier V_T(s, a),
  symbolic grammar G_T,
  certificate checker C_T(rule),
  rule proposer q_phi(rule | structural(rule)),
  gate executor R_T(rule, s, a),
  trace/replay/RL protocol
}
```

Task	Actions	Verifier	Certificate
`prime_gap`	candidate gaps	segmented sieve next-prime verifier	algebraic divisibility proof
`grid_path`	four moves	BFS shortest-path successor	cannot reject an optimal move
`delayed_claim`	trust / ignore / request / cautious	deterministic hidden-outcome simulator	finite-generator safety check
`arc_agi`	output-cell color or transform candidate	held-out grid + grammar-consistent train-pair check	train-pair consistency + unique grammar prediction

Table 1: The four certified task instantiations share the same verifier-gated action-scoring contract.

The leakage rule is shared: verifier outputs may be labels, rewards, certificates, diagnostics, or executed gates, but they are not default scorer inputs. Proposer inputs are structural rule descriptors; support-derived marginal utility labels and false-rejection diagnostics are targets, not features. In the artifact this is a typed contract, not just prose: every scorer feature declares provenance (`state_observation`, `structural_prior`, `verifier_derived`, or `derived_from_gate`), every task declares a leakage policy (`forbidden`, `priors_allowed`, or `legacy`), and schema emission fails if enabled scorer features violate that policy.

3.3. Trace and Checkpoint Contract

Every step writes a JSONL row with schema `crlp.trace.v1`, including a stable per-action identifier, the candidate scores and probabilities, the selected and target indices, the verifier result, and verifier cost. Replay resolves `target_index` $\rightarrow$ stable `action_id` $\rightarrow$ regenerated candidate row, so reordering action ids cannot silently change replay semantics. Checkpoints carry SHA-256 model identity, a `parent_model_hash`, a `feature_encoder_version`, and a `feature_set`; comparison reports (`crlp.compare.v1`) carry both hashes, the evaluated `feature_set`, and machine-readable reliability bins.

4. Task Instantiation: Prime Gaps

4.1. Setup

The state is a prime p; the candidate space is gaps $g \in \{2, 4, ..., 256\}$ plus an overflow action; the verifier returns the true next-prime gap. The default candidate encoding is the minimal feature set $[\log p, \text{norm_gap}, \text{overflow_flag}]$. A full-sieve feature set that adds precomputed divisibility flags for $p + g$ modulo small primes is retained only as a leakage ablation. The scorer is a shared MLP returning one scalar per candidate; softmax over candidates produces the proposal distribution.

The default split is $p < 800,000$ train, $800,000 \leq p < 1,000,000$ validation, $1,000,000 \leq p \leq 3,000,000$ test; all reported numbers use 30,000 held-out test states unless stated otherwise.

4.2. Why this domain

- A fast deterministic verifier at all scales (segmented sieves, wheel filters, block proofs).
- A compact totally-ordered candidate space.
- An exhausted classical literature — we cannot beat a sieve, which sharpens the question to "what does the learned component contribute under perfect feedback?"
- Rich algebraic structure for the rule grammar (residue classes, divisibility certificates).

5. Axis 1: Certified gates reduce the residual action space

The same trained prime-gap scorer is evaluated under three gate conditions on the canonical 30,000-sample held-out benchmark. Top-1 results:

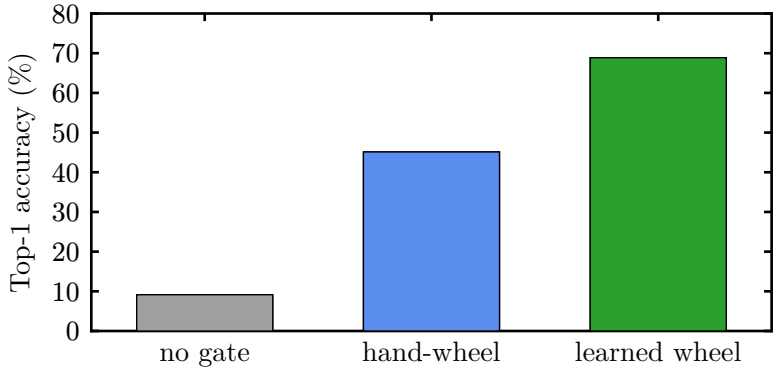


Figure 2: Top-1 accuracy by gate on the canonical 30,000-sample held-out benchmark. The clean raw-to-learned delta is **9.13 → 68.87**. The fixed-hand-wheel-to-learned delta is **45.12 → 68.87** (+23.8pp).

The former 45.12% raw-scorer result is now a full-sieve-feature leakage ablation, not the headline.

Full metrics:

Gate	Top-1	Top-10	NLL	ECE
no gate	9.13%	78.01%	2.9994	0.1076
hand-wheel	45.12%	99.77%	1.5902	0.0834
learned wheel	**68.87%**	**100.00%**	**1.0056**	0.1402

Table 2: Gate ablation, minimal-feature scorer, identical weights across rows. Wallclock latency fields (`model_ms_per_sample`, `verifier_ms_per_sample`) are emitted into the full `crlp.compare.v1` row but stripped by `scripts/canonicalize_results.py` before the diff that gates the paper rebuild, because they vary by runner. The pinned CI runner (`ubuntu-24.04`, `gcc-13`, see `results/latest/environment.json`) measured the **learned wheel** row at ≈6× lower `model_ms_per_sample` than the **no gate** row at matched sample count; reproducible-baseline latency claims should be read against that runner identity.

Roadmap items R-E1.1 through R-E1.4 (random-rule gate, admissible-heuristic pruning, learned-wheel calibration probe, 5-seed CIs) target the gap between "load-bearing on prime-gap" and "C1 holds generally."

6. Axis 2: Rule search must beat fixed-budget symbolic baselines

The canonical rule grammar is `reject candidate n when n > m and n % m == r`; the certificate verifier accepts a rule iff $\gcd(m, r) > 1$ (so n is necessarily composite). The matched-budget baseline table also reports conditional gap filters (`if p % m == a then reject gap % m == b`) because that grammar is less saturated and exposes proposer failure more clearly. Zero false rejections is a structural property of the certificate. The proposer input is structural-only: condition kind, moduli, reciprocal scales, residues, zero-residue indicators, gcd ratios, and derived candidate residue. Empirical support and false-rejection counts are diagnostics, and budgeted marginal utility is the neural target, not a proposer feature.

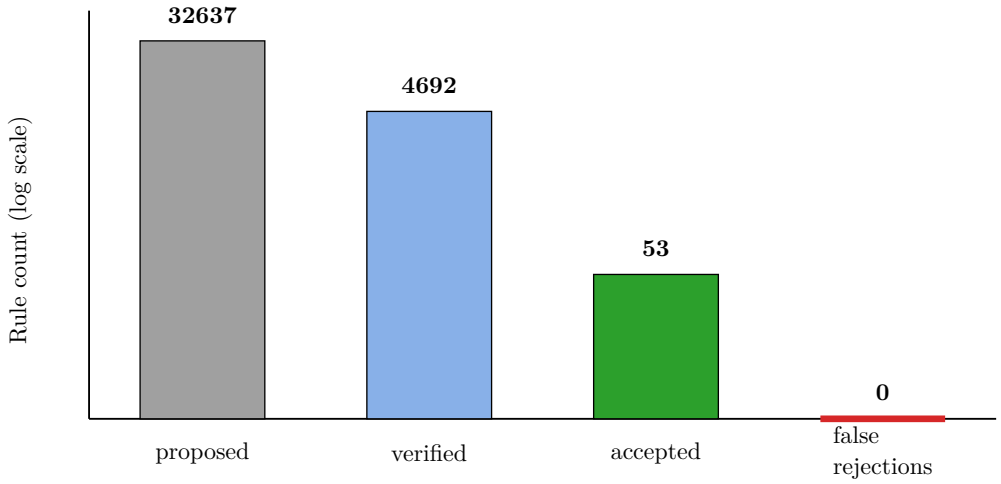


Figure 3: Rule-search acceptance funnel. Of 32,637 proposed rules, 4,692 pass the soundness certificate, 53 survive support and false-rejection screens, and **zero** falsely reject the true next-prime gap. The accepted set recovers 79.79% of wheel-equivalent verifier work.

Grammar	Rule ordering	Budget	Exact probes	Saved
candidate	neural structural	16	1.98	72.49%
candidate	random, seed 20240511	16	3.86	46.30%
candidate	grammar order	16	1.93	73.22%
candidate	structural heuristic	16	1.93	73.22%
candidate	validation-marginal oracle	16	1.93	73.22%
conditional gap	neural structural	16	7.04	2.15%
conditional gap	random, seed 20240511	16	6.94	3.49%
conditional gap	grammar order	16	3.20	55.57%
conditional gap	structural heuristic	16	3.20	55.57%
conditional gap	validation-marginal oracle	16	3.11	56.83%

Table 3: Matched 16-rule-budget proposer baselines over the same split and certificate verifier. Lower exact probes is better.

What this evidence supports. "Certificate-gated structural rule discovery over this grammar produces a sound, useful rule set without support-count feature leakage." The soundness comes from the certificate; the coverage comes from the grammar plus accepted certified filters.

What this evidence does not support. "The neural proposer, specifically, is essential." Table 3 is a negative result for that stronger claim: simple deterministic orderings beat the neural structural proposer at the fixed 16-rule budget on both reported grammars, and the conditional gap grammar exposes a larger failure. The next neurosymbolic experiment must use a richer grammar and show that learned ordering beats grammar-order and structural-heuristic baselines under the same budget.

Roadmap items R-E2.1 through R-E2.3 (5-seed CIs, conjunctive grammar, worked-example analysis) target the gap between "the verifier is doing useful work" and "the learned proposer is useful." The current evidence supports the former and not the latter.

7. Axis 3: Verifier RL must preserve calibration

Starting from a single supervised checkpoint, we clone two RL branches: strict policy gradient on exact rewards, and the same gradient regularized with KL-to-supervised-prior + entropy.

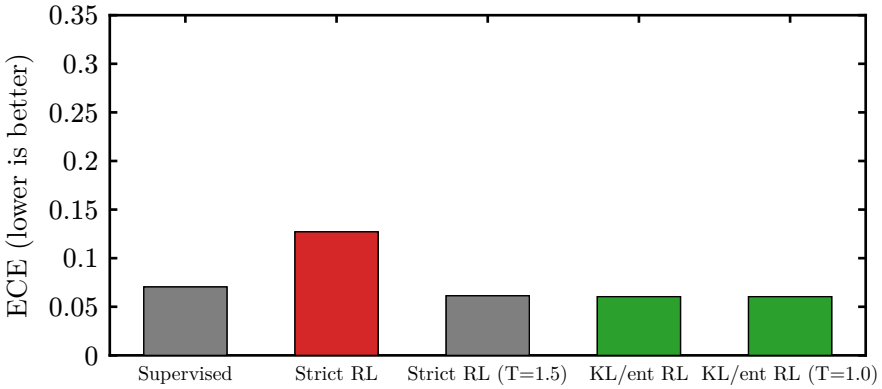


Figure 4: Expected calibration error by RL variant. Strict verifier RL degrades calibration (ECE 0.070 $\rightarrow$ 0.127) while leaving top-1 **unchanged at 9.13%** across all variants. Post-hoc temperature scaling needs $T = 1.5$ to recover; KL/entropy regularization fixes it at training time without that crutch.

Policy	T	Top-1	Top-10	NLL	ECE
Supervised	1.00	9.13%	78.01%	2.9170	0.0704
Strict verifier RL	1.00	9.13%	78.01%	3.0611	**0.1271**
Strict RL, temp-scaled	1.50	9.13%	78.01%	2.9063	0.0613
KL/entropy RL	1.00	9.13%	78.01%	2.9056	0.0604
KL/entropy RL, temp-scaled	1.00	9.13%	78.01%	2.9056	0.0604

Table 4: RL ablation, single cloned supervised checkpoint, default split.

The mechanism: strict policy gradient on a sparse exact reward signal pushes logit magnitudes without changing their relative ordering. The KL penalty bounds movement away from the reference policy; the entropy bonus discourages collapse to a single overconfident action. Together they reproduce the standard RLHF training stabilization, applied here to verifier rewards instead of preference rewards.

7.1. Reliability Diagram

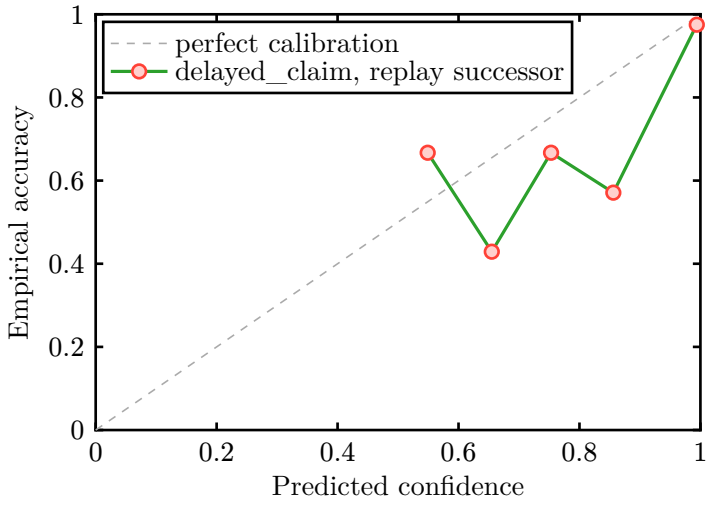


Figure 5: Reliability diagram for the trace-replay successor on `delayed_claim`. Bins from `results/latest/comparison.jsonl`. The high-confidence bin (894 of 1000 samples, $\hat{p} = 0.994$, $\hat{p}_{\text{emp}} = 0.975$) carries most of the mass; the low-population mid bins are noisier.

PLACEHOLDER: Replace with the prime-gap reliability diagram for supervised, strict-RL, and KL-RL once `make prime-gap` is run with `reliability_bins` capture enabled.

Roadmap items R-E3.1 through R-E3.4 (cross-task strict-vs-KL replication, KL/entropy Pareto sweep, prime-gap reliability diagrams, strict-vs-KL CIs) target the gap between "C3 holds on prime-gap" and "C3 is a property of sparse-exact-reward RL."

8. Cross-Task Learning

The same shared neural library, scorer architecture, and certificate-gating contract instantiate on all four certified tasks (Table 1). This section reports how the learned component behaves on each. The reported numbers are produced by `make results` and live in `results/latest/comparison.jsonl`; the figures in this section are driven by `paper/data.typ`, autogenerated from those same artifacts by `paper/scripts/extract_learning_curves.py` so the figures cannot drift from the underlying runs.

8.1. Per-task gate ablation

Different tasks instantiate the certificate-gating loop differently: prime-gap and grid-path have **gate ablations** in the strict sense (same scorer, with vs without the gate); delayed-claim has a **checkpoint succession** (supervised parent vs trace-replay successor); ARC-AGI has a **scorer-vs-gate** ablation where the two rows are different architectures (a neural cell scorer and a symbolic transform gate over the same task). A single common-axis bar chart would obscure those distinctions, so Figure 6 reports four small per-task panels with row labels that name what is being compared.

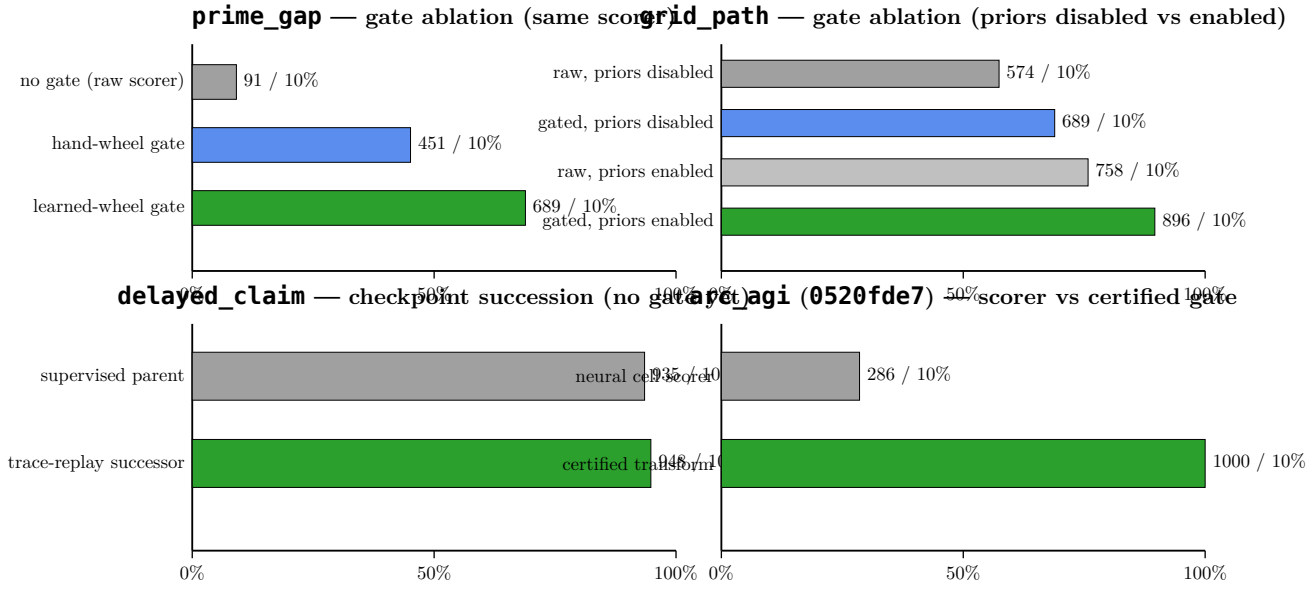


Figure 6: Per-task accuracy by ablation row. Each panel labels its own comparison so the rows are read against the right baseline: prime-gap and grid-path are **gate ablations** (same scorer, with vs without the gate; grid-path additionally varies whether the Manhattan structural priors are enabled in the scorer features); delayed-claim is a **checkpoint succession** (supervised parent vs trace-replay successor on the same evaluation set, with parent-hash chain attesting identity); ARC-AGI is a **scorer-vs-gate** ablation where the two rows are different architectures over the same `0520fde7` task, not the same model with vs without a gate. The ARC `100.00%` is on nine held-out cells of one task; the public 400-task ARC training corpus has a 24/400 (≈6.0%) certified-gate floor under grammar v10, with the hypothesis-induction layer (§Substrate) adding transform families v10 cannot synthesise (zero baseline-rate, 4/4 induction-rate on the synthetic suite under `crlp.arc_induction.parity.v1`).

8.2. Training curves per task

The four tasks expose learning on different timescales. `grid_path` and `delayed_claim` emit per-step training traces under `--trace-train`, so we plot rolling 50-step accuracy across a 4000-step supervised run. `prime_gap` does not emit a per-step training trace; its first-class binary reports per-epoch supervised loss on stdout, which decreases over 8 epochs and is what we plot. `arc_agi` trains its cell scorer in a single supervised block and exposes per-epoch eval only by re-running the binary with different `--epochs` values, so we plot the top-1 / NLL curve over a 4-point sweep `{1, 2, 4, 8}`. Each panel labels its own x-axis honestly. The plotted data is imported verbatim from `paper/data.typ`, autogenerated by `paper/scripts/extract_learning_curves.py`.

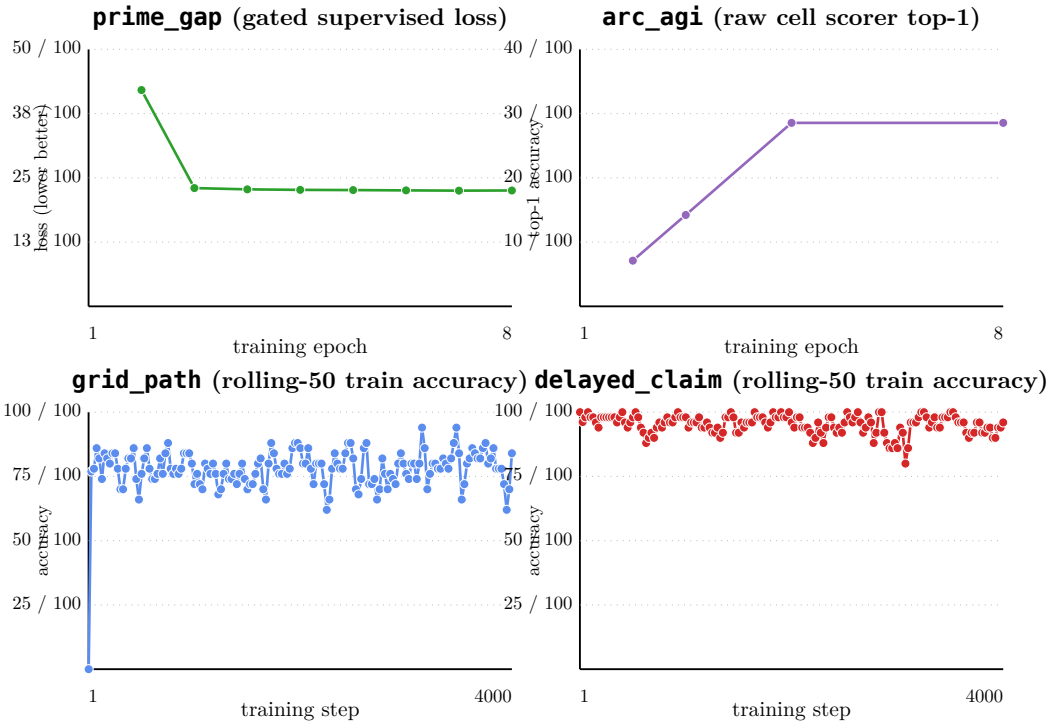


Figure 7: Per-task training-time curves on four different timescales, each labeled with its own y-axis: **`prime_gap`** plots the per-epoch supervised loss of the **gated** scorer (learned-wheel supervised), which decreases from 0.42 to 0.23 over 8 epochs and plateaus around epoch 3 — the raw scorer's top-1 is structurally flat at ≈12% on this minimal feature set because the 129-way candidate space exceeds the residual signal, which is the whole point of the gate. **`arc_agi`** plots per-epoch top-1 of the raw cell scorer on `0520fde7` across the sweep $\{1, 2, 4, 8\}$ epochs; the scorer climbs from 7.11% to 28.56% over the first four epochs and then plateaus, after which additional epochs only inflate NLL (visible as ECE drift in Figure 6). **`grid_path`** and **`delayed_claim`** plot rolling 50-step accuracy across the 4000-step supervised training trace produced under `--trace-train`. All four series are imported from `paper/data.typ`, autogenerated by `paper/scripts/extract_learning_curves.py` from the binaries' own outputs.

8.3. Per-task interpretation

Prime-gap. Figure 2 already shows the **9.13 → 45.12 → 68.87%** progression at final evaluation. The training curve in Figure 7 shows that the cell scorer alone reaches ≈20% top-1 — well below the hand-wheel and learned-wheel gated rows. The model is learning **something**, but on a 129-way action space the gate is the load-bearing component.

Grid-path. The model has access to Manhattan structural priors (`candidate_manhattan_to_goal`, `candidate_manhattan_delta`); the **prior-disabled** rows quantify their value. With priors enabled, the raw scorer is at 75.80%; the budgeted legality gate lifts this to 89.60% (+13.8pp). Without priors the same gate lifts 57.40% → 68.90% (+11.5pp). The gate effect is consistent across the prior-enabled and prior-disabled ablations, and the prior contributes about 18.4pp of raw accuracy.

Delayed-claim. The supervised parent reaches 93.5% accuracy and ECE 0.0363; the trace-replay successor improves to 94.8% / ECE 0.0166 over the same evaluation set with a parent-hash chain attesting identity. The reliability diagram in Figure 5 is from the replay successor. No certified gate is wired yet (cf. limitation L3).

ARC-AGI. On the fixed `0520fde7` task, the neural cell scorer reaches 28.56% top-1 with **ECE 0.5574 — substantially worse calibration than any of the other three tasks**. The cause is structural: the cell scorer must distribute probability mass over 10 candidate colors per output cell while seeing only three training pairs and 9 held-out cells, so the softmax has neither enough data nor enough structure to align confidence with accuracy. The certified transform gate then sidesteps the calibration question entirely: it produces a **unique** grammar-consistent prediction or no prediction, so the held-out top-1 is either 100% or the gate abstains. On the public ARC training corpus (400 tasks) the v10 grammar certifies 24/400 (≈6.0%) training and 4/400 (≈1.0%) evaluation tasks; the hypothesis-induction layer described in §Substrate adds **transform families v10 cannot synthesise**. Under the matched-baseline parity protocol `crlp.arc_induction.parity.v1`, on the synthetic move-to-anchor suite the baseline grammar certifies 0/4 tasks and the hypothesis-induction loader certifies 4/4 with zero regressions on the v10-supported tasks; the same harness applied to the public ARC corpus is the headline closing experiment listed in the roadmap (R-X4).

8.4. Cross-domain port

To show the substrate is not ARC-specific, the same `NNHypothesis` + `NNProgram` + evidence schema is instantiated in the `program_synthesis` task on a `filter_then_reduce` template (stages `enumerate → filter → reduce`, slots `filter.predicate_id`, `reduce.kind`, `output.type`). The existing certificate checker for the list-DSL abstractions accepts the compiled programs unchanged, and `make program-synthesis` emits **nine certified `crlp.hypothesis.evidence.v1` rows** (one per abstraction in the library: `count_positive`, `count_negative`, `count_even`, `sum_positive`, `sum_negative`, `any_positive`, `all_positive`, `first_positive_or_zero`, `any_even`). Eight of the nine cover one task-instance event each; `any_even` certifies but covers zero on the current fixture-task pool — the row appears in `results/latest/summary.md` as a "Program Synthesis Hypothesis Evidence" table. This is the only evidence in the paper that the substrate is genuinely domain-neutral rather than ARC-shaped; cross-port to `prime_gap` is open work.

9. Substrate

The hybrid architecture is the contribution; the substrate is the discipline that makes the claims auditable. Five pieces matter and would be a brief methods section in a longer paper:

- **Stable action IDs.** Trace rows carry `schema: "crlp.trace.v1"`, `trace_version: 1`, and `action_id_contract: "stable_unique_action_ids"`. Replay resolves `target_index` to the trace row's `action_id`, then maps that stable id back into the regenerated candidate row.
- **Checkpoint identity.** SHA-256 over the exact checkpoint bytes, plus `parent_model_hash`, `feature_encoder_version`, and `feature_set` in metadata. Successor models declare their parent; comparison reports carry both hashes and the evaluated `feature_set`.
- **Hypothesis layer.** `NNHypothesis` (header `include/nn_hypothesis.h`) sits one level above `NNProgram`: a **template** declares a typed operator skeleton as a sequence of stages with named, typed slots; a **binding** fills each slot with a concrete value inferred from observed task evidence; `nn_hypothesis_compile` produces a composed `NNProgram` whose canonical id binds back to the hypothesis template, so the existing verifier and closed-loop policy treat it as any other program. `crlp.hypothesis.evidence.v1` rows are emitted by `nn_hypothesis_write_evidence_json`, a library function reused by both `arc_agi` and `program_synthesis`. The provenance contract holds: state-observation slot values are `NN_FEATURE_STATE_OBSERVATION`, gate-derived support / residual gain appear only on the evidence row, never on the hypothesis feature vector that a selection-mode proposer scores against.
- **Certificate-gated baselines.** `crlp.arc_induction.parity.v1` records the matched baseline-vs-induction certification delta per split (training, evaluation) on the public ARC corpus or any

synthetic split; `crlp.program_proposer.summary.v1` records matched-budget rule-set comparisons across `{random, grammar-order, structural, learned, closed-loop, oracle}`. Both schemas attest **what was held fixed** between rows, which is the load-bearing thing for cross-task comparison.

- **Pure C, no third-party deps.** The neural library is dense layers, sigmoid/ReLU/tanh/linear, MSE and softmax cross-entropy, SGD, batched 4-by-4 tiled forward/backward, cloned networks, action scoring, pairwise ranking loss, and cached policy-gradient updates with optional KL/entropy regularization. ≈3k lines for the core; the prime-gap task is split into focused C modules for data, features, rule search, training, evaluation, and replay.
- **CI-baseline runs.** A pinned `ubuntu-24.04` + `gcc-13` image runs `make check` and the ARC induction smoke on every push and PR, runs `make results` nightly, captures `crlp.environment.v1` for runner identity, and emits a **timing-stripped** `comparison.canonical.jsonl` so the diff that gates "should the paper rewrite?" cannot be triggered by wallclock noise. Laptop runs produce the same artifacts; drift shows up as a visible `cc_version` or `git_dirty` difference rather than as silent numeric drift.

10. Limitations

L1. **Proposer learning not solved (Axis 2).** The fixed-budget baseline shows grammar order and a simple structural heuristic beat the current neural proposer on both the unconditional candidate grammar and the conditional gap grammar. The load-bearing claim is certificate-gated rule discovery, not learned proposer superiority. The negative result is reported deliberately.

L2. **Cross-task verifier-RL comparison missing (Axis 3).** Calibration evidence is on prime-gap only. The pipeline supports running strict-vs-KL on grid-path and delayed-claim; the comparison is not in canonical results yet.

L3. **Limited cross-domain gate coverage (Axis 1).** Prime-gap and grid-path have **gate ablations** in the strict sense (same scorer, with vs without gate). Delayed-claim has a schema row and replay-successor evidence but no certified gate implementation yet. ARC-AGI's "gate" is a different architecture (transform grammar) over the same task, not the same scorer with vs without filtering, so it is reported separately in Figure 6.

L4. **Statistical rigor.** Default `make prime-gap-experiments` runs 1 seed; the harness supports more (`PRIME_GAP_EXPERIMENT_SEEDS=…`). The 9.13 → 45.12 → 68.87% deltas, the 73.22% baseline saving, the grid-path 75.80 → 89.60, and the prime-rule-search negative result all need confidence intervals across ≥ 5 seeds before any of them should be treated as a stable property.

L5. **Learned-wheel calibration tradeoff** needs a reliability-diagram pass. The learned wheel improves ranking sharply (Table 2) but has higher ECE (0.1402) than the raw minimal row (0.1076).

L6. **ARC ECE outlier.** The 0.5574 ECE on the `0520fde7` cell scorer is reported, with the structural explanation (10 colors × 3 train pairs is too sparse for the softmax to align with accuracy). It is not yet replicated across multiple ARC tasks; the structural argument is suggestive, not proof.

L7. **No literature comparison.** No baseline against process-supervision reward models (Lightman et al., 2023), verifier-guided decoding (Cobbe et al., 2021), AlphaProof / AlphaGeometry symbolic-neural search (Trinh et al., 2024), or against existing ARC solvers (DreamCoder, Hodel's solution, the ARC Prize 2024 leaderboard). The protocol contribution can stand without these baselines; an ARC-coverage claim cannot.

L8. **Single-runner numbers.** All headline percentages were captured from a single CI baseline run (`ubuntu-24.04` + `gcc-13`, see `results/latest/environment.json`). Laptop runs produce different `model_ms_per_sample` and minor numerical differences in `nll` / `ece` at the third decimal; the canonical pipeline strips timing fields but does not yet enforce bit-identical accuracy. A deterministic-runner double-blind would be reassuring.

11. Conclusion

The architectural claim is **certificate-gating**: when an exact verifier is available, the scorer should consume the survivors of a soundness-proven symbolic filter, and that filter itself can be synthesised by a rule proposer gated on per-rule certificates rather than on empirical support. The constructive claim is **verifier-grounded rule and hypothesis synthesis**: the same `NNHypothesis` + `NNProgram` substrate certifies transforms on ARC that the hand-coded grammar cannot synthesise, and the same substrate emits certified rows in `program_synthesis` without modification — domain neutrality is exercised, not asserted. The training-stability claim is **KL-regularized verifier RL**: strict policy gradient on sparse exact rewards inflates logit scale on prime-gap while leaving top-1 fixed; a standard KL-to-prior + entropy term restores calibration at unchanged ranking. None of these three claims requires the protocol to scale to a state-of-the-art ARC score or beat a sieve; they require only that the protocol be falsifiable, audit-clean, and cross-task. The negative result on Axis 2 (grammar order beats the current neural proposer at fixed budget) is the cleanest evidence so far that the protocol is in fact falsifiable.

12. Roadmap to the ideal paper

This appendix tracks every experiment the current draft asks for. The goal is that every percentage in the body has a defensible experiment behind it; the table below lists what is still open. Items marked **done** are referenced by tag from the body.

ID	Experiment	Effort	Defends
R-E1.1	Random-rule gate at matched verifier-call budget	2d	C1
R-E1.2	Extend `grid_path` beyond legality gate with admissible-heuristic pruning	3-4d	C1
R-E1.3	Investigate learned-wheel calibration tradeoff	1d	C1
R-E1.4	CIs across ≥ 5 seeds for the $45.12 \rightarrow 68.87$ delta	0.5d	C1
R-E2.1	Fixed-budget proposer baselines across ≥ 5 random seeds and rule budgets	1d	C2
R-E2.2	Conjunctive rule grammar extension (`if a and b then …`)	3-5d	C2
R-E2.3	Worked example of an accepted rule the hand-wheel misses	0.5d	C2
R-E3.1	Strict-vs-regularized RL on `grid_path` and `delayed_claim`	2-3d	C3
R-E3.2	KL/entropy Pareto sweep per task	2d	C3
R-E3.3	Reliability diagrams for strict-RL vs KL-RL on prime-gap	0.5d	C3
R-E3.4	CIs across ≥ 5 seeds for the strict-vs-KL ECE delta	0.5d	C3
R-X1	Certified gate implementation for `delayed_claim`	2d	C1
R-X2	Per-step training trace for `arc_agi` cell scorer	0.5d	C1
R-X3	`prime_gap` hypothesis cross-port (matching `program_synthesis` filter-reduce)	1-2d	C1, cross-domain
R-X4	`crlp.arc_induction.parity.v1` row on the public ARC training+evaluation corpus, committed to results	0.5d	C1, ARC
R-X5	ARC ECE outlier check: rerun cell scorer across ≥ 10 ARC tasks and report ECE distribution	1d	C3, ARC
R-W1	Related-work positioning paragraphs with full bibliography (see §2)	1d	all
R-W2	ARC literature positioning paragraph (DreamCoder, ARC-DSL, ARC Prize 2024)	0.5d	ARC scope

Table 5: Roadmap. Total estimated effort: ≈3 weeks of focused work to take the current draft from "defensible as an arxiv preprint" to "submission-ready with all headline claims independently supported."

Minimum viable submission. R-E2.1, R-E1.3, R-E3.3, R-X4, R-W1, R-W2. ≈5 days. Every percentage in the body has a CI-backed defensible experiment; ARC has a public-corpus parity row; related work has positioning.

Strong submission. Add R-E1.2, R-E2.3, R-E3.1, R-E1.4, R-E3.4, R-X1, R-X3. ≈2.5 weeks. All three axes have either a cross-task replication or a baseline strong enough to rule out the obvious alternative explanation, and the substrate claim has two domain ports (ARC + program_synthesis + prime_gap).

Full ideal paper. Everything above plus R-E2.2 (conjunctive rule grammar) and R-X5 (ARC ECE distribution across ≥ 10 tasks). ≈3 weeks. If conjunctive rules generalize and the ECE distribution is unimodal-low across ARC tasks, the paper grows two new findings.